

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192511053
Name	B Santhosh Kumar

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

*I **B Santhosh Kumar/192511053**(Name / Reg No) certify that this submission is my original work and that I have adhered*

to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

B Santhosh Kumar

Signature of the student:

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

PROPOSED SYSTEM

Social Sense AI – Distributed Multi-Agent Platform for Real-Time Urban Intelligence

Problem Chosen

Large cities continuously generate data from social-media posts, emergency reports, traffic updates and public messages. The problem is not only collecting this data but processing it quickly enough to identify important events and decide which response should happen first.

For example, during heavy rainfall, thousands of social-media posts may appear about waterlogging, traffic blockage and stranded people. Processing these posts one by one on a single machine would become difficult as the volume increases.

My proposed system uses **PySpark and RDDs for distributed processing** and a group of intelligent agents for event analysis, priority estimation and resource allocation.

A similar research direction has already been demonstrated in a real-time urban analytics system that used Apache Spark to process streaming social-media data for topic modelling, sentiment analysis and rapid event detection during disasters and public-health events.

1. END-TO-END SYSTEM DESIGN

1(a) System Architecture

The proposed architecture has six main stages:

Main components

Component	Role in my system
Social Media Sources	Collect public posts, hashtags and location-related messages
Kafka / Streaming Layer	Receives incoming messages continuously
PySpark	Processes data in parallel
RDD Layer	Performs filtering, mapping, grouping and aggregation
AI/ML Layer	Detects topics, sentiment and event severity

Agent Layer	Decides what action should be taken
Recommendation Engine	Produces alerts, priority and suggested response
Cloud Storage	Stores processed historical data

The design follows the main advantage of Spark RDDs: data can be partitioned across a cluster and processed in parallel, while lost partitions can be recomputed using lineage rather than requiring every intermediate result to be replicated.

Proposed Data Flow

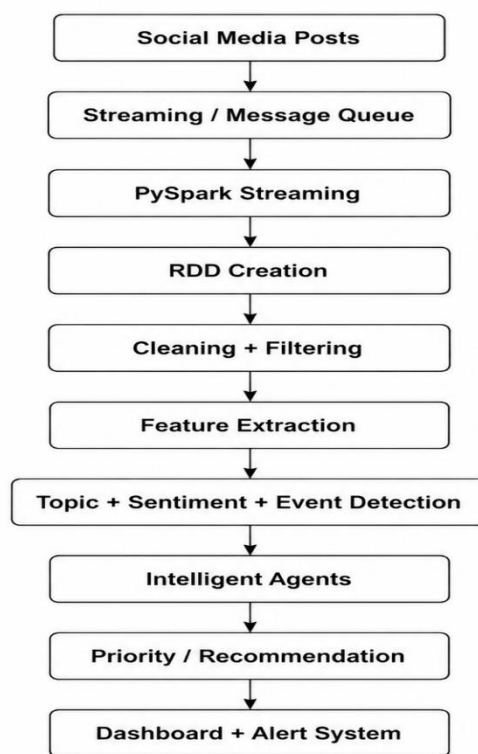


Figure 1: Proposed Data Flow

Example

Suppose these posts arrive within a few seconds:

- “Water has entered the road near Tambaram.”
- “Traffic completely stopped near the bridge.”
- “People are stranded near the bus stand.”

The Spark layer processes these messages together. The AI layer detects:

Topic:Flood/Waterlogging

Sentiment:Negative/Emergency

Location:Tambaram

Severity: High

The response agent can then increase the priority and recommend sending traffic-control or rescue resources.

1(b) RDD Operations

I would use the following RDD operations:

Operation	Use
map()	Convert raw messages into structured records
filter()	Remove irrelevant or empty posts
flatMap()	Extract individual words/hashtags
reduceByKey()	Count events or hashtags
groupByKey()	Group messages by location
join()	Combine social-media events with location/zone data
distinct()	Remove duplicate messages
cache()	Keep frequently reused data in memory
count()	Count messages/events

The reason for using RDDs is that these operations can be executed across partitions instead of processing the complete dataset on one machine. RDDs were specifically designed for parallel, fault-tolerant computation on clusters.

1(c) Scalability

The system should not depend on a single Spark worker.

A possible deployment is:

- 1 Spark driver
- 4–20 worker nodes depending on workload
- Multiple partitions for incoming data
- Distributed cloud storage
- Separate agent services

When social-media traffic increases, more Spark workers can be added.

For example:

Normal traffic → 4 workers

Major disaster → 12 workers

This allows the processing capacity to increase according to demand.

Spark Streaming research also demonstrated that distributed streaming can scale while providing fault recovery and low latency.

2. DISTRIBUTED INTELLIGENT AGENT SYSTEM

2(a) Agent Design

Instead of creating one large AI agent, I would divide the work among specialized agents.

Agent	Responsibility
Data Agent	Checks incoming data quality
Event Agent	Identifies possible incidents
Sentiment Agent	Determines public reaction/severity
Location Agent	Groups incidents by geographical area
Resource Agent	Checks available response resources
Decision Agent	Selects the final recommended action

This is useful because each agent has a specific job.

For example, during a flood:

- **Event Agent:** Detects flooding events.
- **Location Agent:** Identifies the affected area.
- **Resource Agent:** Checks the availability of rescue teams.
- **Decision Agent:** Decides which rescue team should be assigned first.

The agents communicate their results rather than each agent processing the complete problem independently.

Research on multi-agent optimization shows that multiple agents can cooperate through local decisions and communication to solve large-scale optimization problems.

2(b) Large-Scale Optimization Problem

The optimization problem in my system is:

Assign limited emergency resources to the highest-priority incidents while minimizing response time.

For example:

There are:

- 10 incidents
- 4 rescue teams
- 3 ambulances
- 2 traffic-control teams

The system must decide which resource should go to which incident.

Agent objective

Each agent tries to improve:

- Response time
- Incident coverage
- Resource utilization
- Priority handling

A simplified objective is:

Maximize = Incident Priority – Response Time – Resource Cost

2(c) Agent Collaboration



Figure 2– Agent Collaboration.

This is more practical than making every agent independently decide the final action.

Recent work on multi-agent reinforcement learning for resource allocation similarly emphasizes decentralized agents, dynamic environments and coordination when resources are limited.

3. BIG DATA AI PIPELINE DESIGN

3(a) Complete Pipeline

Stage 1 – Data ingestion

Incoming social-media messages are collected continuously.

Stage 2 – Cleaning

Remove:

- Empty posts
- Duplicate posts
- URLs when not required
- Spam messages
- Irrelevant content

Stage 3 – Feature extraction

Useful features can include:

- Message text
- Location
- Timestamp
- Hashtags
- Sentiment score
- Event keywords
- Number of mentions

Stage 4 – Distributed processing

3(b) Feature Engineering

I would convert raw text into features such as:

Feature	Example
Flood keyword count	4
Emergency keyword count	3
Negative sentiment	0.91
Location	Tambaram
Post frequency	250/min
Event severity	High

These features can then be used by the AI model and decision agents.

3(c) Model Training

For large-scale machine learning, I would use **Spark MLlib**.

MLlib was specifically developed as Spark's distributed machine-learning library and supports classification, regression, clustering and other learning tasks. Its integration with Spark allows machine-learning pipelines to work with distributed data.

For this system, a possible model is:

Random Forest / Logistic Regression → Event Classification

Classes could be:

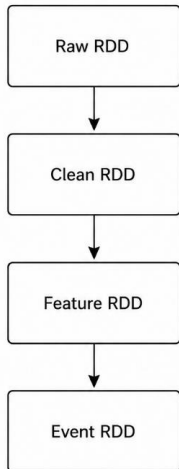
- Normal
- Traffic
- Flood
- Fire
- Medical Emergency
- Other

The historical dataset can be divided into training and testing data.

3(d) Fault Tolerance

RDDs use **lineage**.

For example:



If a worker loses one partition, Spark can reconstruct that partition using the transformations that produced it.

This is one of the main ideas introduced in the original RDD research paper.

For frequently reused RDDs, I would use:

`features.cache()`

This avoids recomputing the same intermediate data repeatedly.

4. REAL-TIME DECISION-MAKING SYSTEM

4(a) Real-Time Scenario

Consider a sudden flood in a city.

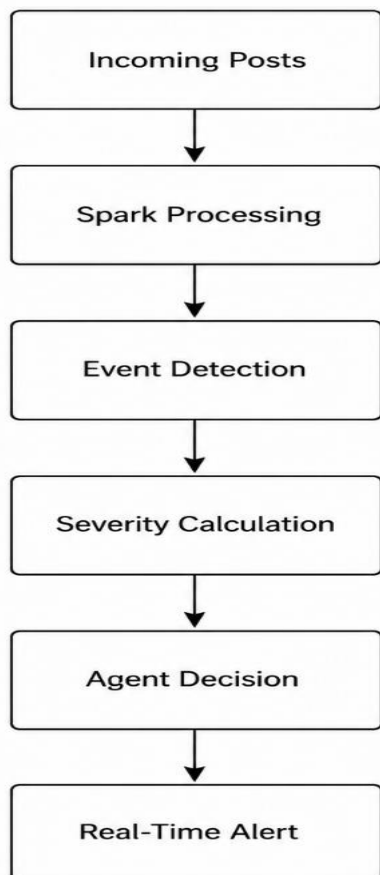
At 10:00 AM:

- Social posts suddenly increase.
- People report waterlogging.
- Traffic complaints increase.
- Emergency keywords appear.

The system should not wait until the end of the day to analyse these posts.

The streaming layer continuously sends data to Spark.

The process becomes:



The Spark Streaming research introduced D-Streams specifically for real-time applications requiring parallel processing and fault recovery.

4(b) Decision Logic

I would use a simple priority score initially:

$$\text{Priority Score} = 0.4 \times \text{Severity} + 0.3 \times \text{Post Frequency} + 0.2 \times \text{Affected Population} + 0.1 \times \text{Sentiment}$$

Example:

Factor	Value
Severity	9/10
Post frequency	8/10
Affected population	7/10
Negative sentiment	9/10

The resulting score would classify the event as **High Priority**.

The Decision Agent then checks resource availability.

4(c) Agent Decision

Example:

Incident: Flood

Location: Zone B

Severity: High

Available:

Rescue Team 1 → 12 km away

Rescue Team 2 → 4 km away

Rescue Team 3 → Busy

Decision:

Assign Rescue Team 2

This makes the system useful beyond simply detecting an event.

It converts:

Data → Information → Decision → Action

5. CLOUD-BASED DISTRIBUTED AI APPLICATION

5(a) Cloud Architecture

I would deploy the system using a cloud cluster.

A possible structure is:

Layer	Technology
Data source	Social media APIs
Streaming	Kafka / Spark Streaming
Processing	PySpark
Storage	Cloud object storage
ML	Spark MLlib
Agent services	Python
API	FastAPI
Dashboard	React
Deployment	Docker + Kubernetes
Monitoring	Prometheus / Grafana

Spark can run on standalone clusters, Hadoop, Kubernetes and cloud environments, making it suitable for this type of distributed deployment.

5(b) Dynamic Resource Allocation

The cloud should not keep the same number of workers active all the time.

Example:

Situation	Workers
-----------	---------

Normal traffic	4
High event volume	8
Major disaster	16
Event finished	Scale back to 4

This reduces unnecessary resource usage.

The Resource Agent monitors:

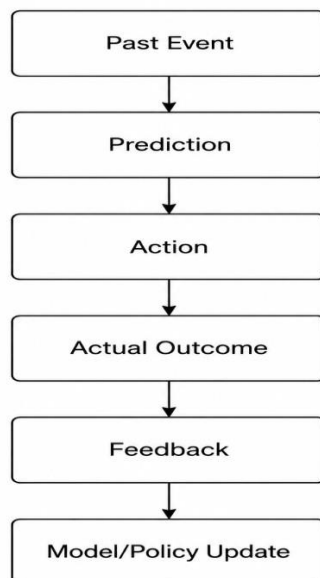
- CPU utilization
- Memory utilization
- Incoming message rate
- Processing latency
- Queue length

If the workload increases, the agent requests additional cloud resources.

5(c) Adaptive Learning

The system stores historical incidents.

For example:



This allows the system to improve over time.

For more advanced deployment, the Resource Agent could use reinforcement learning to select scaling actions.

For example:

Action = Add 2 workers

Reward:

+10 if latency decreases without excessive cost.

-10 if the system spends resources without improving latency.

Multi-agent RL has already been studied for cloud resource allocation, including dynamic allocation/release of resources, load balancing, energy use and fault tolerance.